

Lab 1: Intro to Postgres

These instructions assume that you have the 345 dev container set up and running. If not, you can follow the instructions in the file [Installing and Operating the CS345-DevContainer](#) before starting this lab.

Important note: *The dev container is not required to complete this or any of the other labs! You can simply install the software directly (in the case of this lab, postgres). You can find instructions for installing and using Postgres in our Google Drive.*

Content Links:

1. [Getting started](#)
2. [Part 1: Creating the projects database](#)
3. [Part 2: Creating the imdb database](#)
4. [Questions to Answer and Submit](#)
5. [References](#)
6. [How to Submit to Gradescope](#)

Objectives:

- Launch the psql shell.
- Work with basic psql commands.
- Create/delete a database.
- Create database structure and data load via executing an sql file.
- Execute a basic SQL query.
- Submit files to Gradescope

The steps in this lab will provide you with some basic skills and best practices that will be useful as you progress through this course and beyond. It is important to always check that some action happened as you expected. Error messages often contain useful information. Use a search engine or AI tool to investigate error messages by copying and pasting them into the search box, and always preface with “postgres”. Keep the files you are working with well organized- this really helps!

You will submit three sql files to the Lab 1 Gradescope assignment, one for each question.

Getting started

You will be working with the Postgres database server and management system for this course. The interface we will use for postgres is psql, a shell interface that is used for creating databases, running sql script files, querying data and most other tasks.

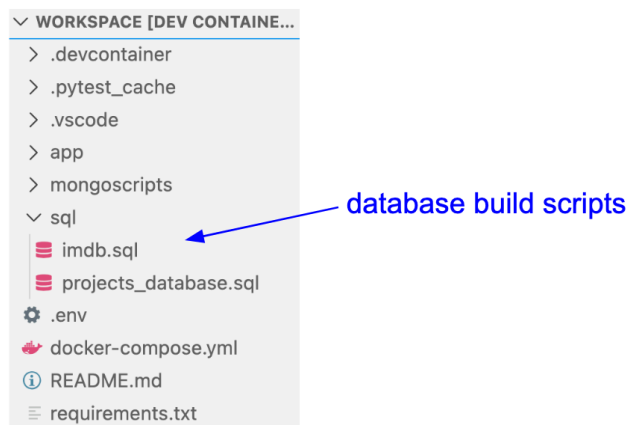
In this lab, you will first create the **projects_database** and then the **imdb** database. There are three questions to answer with SQL statements on the imdb database.

The code to build the structure and load data for these databases are in the files:

imdb.sql and **projects_database.sql**

(Note: the imdb file is very large ~690MB)

The **projects_database.sql** file is included with the dev container file. It is located in the sql folder:



The **imdb.sql** file can be downloaded separately, and should be placed in the sql folder. In the future, place any build scripts in the sql folder.

You can find both **projects_database.sql** and **imdb.sql** in the [directory for this assignment](#).

Part 1: Creating the projects_database

Start a psql shell in VSCode:

```
vscode → /workspace $ psql "$DATABASE_URL"
psql (17.6 (Debian 17.6-0+deb13u1), server 16.10 (Debian 16.10-1.pgdg13+1))
Type "help" for help.

postgres=#
```

Create a new database called projects_database. Do this by executing the SQL create database statement you see below (note that SQL is not case sensitive). After you have created the new database, you can connect to it with the following shell command: \c projects_database.

```
postgres=# create database projects_database;
CREATE DATABASE
postgres=# \c projects_database;
You are now connected to database "projects_database" as user "postgres".
projects_database=#
```

SQL statements in psql are followed by a semicolon, while shell commands are not. Shell commands begin with the backslash (See [Table 1: Some psql shell commands](#), [Table 2: psql prompts](#), and [Table 3: Some SQL statements](#) for reference).

You can see all of the databases in your postgres server by executing the \l command (backslash lowercase l, the l is for list):

```
postgres=# \l
```

You can see the newly created projects_database in the list:

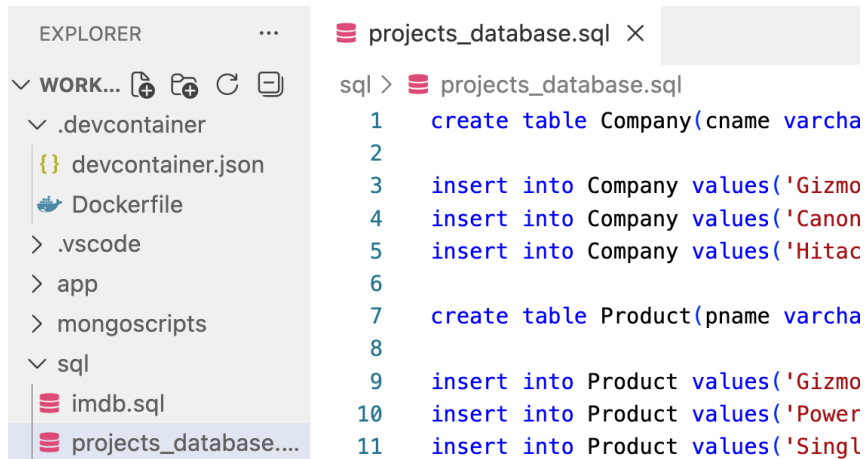
List of databases									
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges	
postgres	postgres	UTF8	libc	en_US.utf8	en_US.utf8				
projects_database	postgres	UTF8	libc	en_US.utf8	en_US.utf8				
template0	postgres	UTF8	libc	en_US.utf8	en_US.utf8			=c/postgres	+
								postgres=Ctc/postgres	
template1	postgres	UTF8	libc	en_US.utf8	en_US.utf8			=c/postgres	+
								postgres=Ctc/postgres	
(4 rows)									

Postgres is the default database. Template databases are used by the system when new databases are created. You can exit the list view by executing the \q command if needed.

Now, your projects_database has been created, but it does not yet have any data or tables. You can see the tables (called relations) in your database with the \dt command:

```
[projects_database=# \dt
Did not find any relations.
projects_database=# ]
```

Next, you will create the database tables and data by executing SQL statements written in an .sql script file. Before doing that, examine the SQL statements that will be executed in the file **projects_database.sql** by clicking on the file in the explorer VSCode:



```
EXPLORER
WORK...
  .devcontainer
    devcontainer.json
    Dockerfile
  .vscode
  app
  mongoscripts
  sql
    imdb.sql
    projects_database.sql

projects_database.sql
sql > projects_database.sql
1 create table Company(cname varchar(50))
2
3 insert into Company values('Gizmo')
4 insert into Company values('Canon')
5 insert into Company values('Hitac')
6
7 create table Product(pname varchar(50))
8
9 insert into Product values('Gizmo')
10 insert into Product values('Power')
11 insert into Product values('Singl')
```

You can see statements that create tables and insert data into the tables. You will work with these types of statements in the next lab.

To execute the code in the projects_database.sql file, use the \i command and the path to the projects_database.sql file (Remember that you placed the sqlscript files in the sql folder at the beginning of the lab). Make sure you are connected to the projects_database and not postgres. You can tell from the name in the prompt.

You should see this output after executing the \i command:

```

projects_database=# \i sql/projects_database.sql
CREATE TABLE
INSERT 0 1
INSERT 0 1
INSERT 0 1
CREATE TABLE
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
CREATE TABLE
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
CREATE TABLE
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
projects_database=#

```

If you get an error message “No such file or directory” you have probably entered an incorrect path or filename.

If you see an error about executing the code, then you will need to delete the database, recreate it and then try loading the file again. You need to delete the database because some of the code might have been executed and you want to run the entire file with no existing tables or data.

Show tables:

If the script **built with no errors**, execute the \dt command to see a list of the tables in the projects_database database:

```

projects_database=# \dt
      List of relations
Schema |   Name   | Type  | Owner
-----+-----+-----+-----
public | company  | table | postgres
public | employee | table | postgres
public | product  | table | postgres
public | projassign | table | postgres
public | project  | table | postgres
(5 rows)

```

If there were errors, **delete** the projects_database with the DROP DATABASE statement (you can also do this just for practice if you wish).

```
[projects_database=# DROP DATABASE projects_database;  
ERROR:  cannot drop the currently open database  
projects_database=#
```

Note: you have to **disconnect** from the database you want to delete before you can delete it. You can connect to the default database named postgres and then delete the projects_database.

```
[projects_database=# \c postgres  
You are now connected to database "postgres" as user "postgres".  
[postgres=# drop database projects;  
DROP DATABASE  
postgres=#
```

Now, recreate the projects_database and run the script file carefully following the steps above.

Note on prompts:

If you see a prompt other than =#, it usually means you have a missing character, such as a semicolon or a quote ([see Table 2](#)). Type the missing character and the normal prompt will return:

```
postgres=# drop database projects_database  
postgres-#  
postgres-#  
postgres-# ;  
DROP DATABASE  
postgres=#
```

After successfully creating the projects database and executing the projects_database.sql file, your projects database now has a few tables and some data that you will query in the next step.

Query the projects_database

[Table 3](#) contains some simple SQL queries. Execute a query that returns all rows from the Employees table. Note that the * symbol means “all columns”. Don’t forget the semicolon.

You should see this result:

empid	name	phone	managerid
1	John	555-1234	5
2	Alice	555-4321	3
3	Peter	555-2314	5
4	Cecilia	555-3241	1
5	James	555-4231	

Part 2: Creating the imdb database

Following the same procedure for creating and populating the projects database, create the imdb database, connect to it, then execute the script in the imdb.sql file with the \i command.

Remember that you have to connect to the imdb database (with the \c command) after you create it before you can execute the file.

Note that this is a much larger database than the projects_database. Although this database will be used later in the course, creating it now will save time later on.

After the file has finished, check on the tables in the imdb database. You should see:

```

      List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | actor           | table | postgres
public | casts           | table | postgres
public | directors       | table | postgres
public | genre           | table | postgres
public | movie           | table | postgres
public | movie_directors | table | postgres
(6 rows)

imdb=#
```

Questions to Answer and Submit

The following three questions will ask you to write and execute queries on the **imdb database**. You will submit your answers to these questions in separate .sql files which you will submit to Gradescope.

NOTE: For all questions, you will submit .sql files only, one file for each question. We suggest using VSCode to create new sql files for each question and keeping them in lab folders. You can use a plain text file for SQL, but ensure utf8 encoding and the .sql extension, e.g., Q1.sql. Do not use a rich-text editor, such as Word, for SQL. Do not copy and paste from a web page. It's your responsibility to ensure that the file you submit can be parsed by the autograder.

A note of caution: When you are dealing with a large database, such as imdb, you may want to limit the result set to avoid a very large number of rows returned.

Q1: Write a query that returns 15 rows from the actor table. Use the LIMIT keyword. Enter your select query (**not the result**) in a file named **Q1.sql**. Submit this file to the Lab 1 assignment in Gradescope.

Q2: Write a query that **returns the number of rows** in the movie table (this will be a large number). Enter your query in a file named **Q2.sql**. Submit this file to the Lab 1 assignment in Gradescope.

Q3: Write a query that returns 10 rows (using the LIMIT keyword) with only the title and year for movies. You might want to first execute a query or shell command to see the exact column names to use in your query. Enter your select query (**not the result**) in a file named **Q3.sql**. Submit this file to the Lab 1 assignment in Gradescope.

References

Table 1: Some psql shell commands

Help	\? (for psql commands) or \h (for sql)
List all databases.	\l
Switch/connect to a specific database.	\c <i>databasename</i>
View all tables in a database.	\dt
View columns/constraints in a table.	\d <i>tablename</i>
Execute SQL script from file.	\i <i>path/filename.sql</i>
Quit / go to the end of a long output.	\q

Table 2: psql prompts

postgres=#	a fresh prompt waiting for the start of a new command
postgres-#	is the result of clicking enter after typing a command that does not end with a semicolon
postgres(#	indicates an open parenthesis in the statement
postgres'#	indicates an unmatched single quote in the statement
postgres"#	indicates an unmatched double quote in the statement

Table 3: Some SQL statements

create a database.	CREATE DATABASE <i>databasename</i> ;
delete a database.	DROP DATABASE <i>databasename</i> ;
get all data from a table	SELECT * FROM <i>tablename</i> ;
get N rows from a table, where N is an integer.	SELECT * FROM <i>tablename</i> LIMIT N;
get the number of rows	SELECT COUNT(*) FROM <i>tablename</i> ;

from a table.	
get only the columns in the <i>columnnames</i> list.	<code>SELECT <i>columnnames</i> FROM <i>tablename</i>;</code>

SQL is not case sensitive, but we often show SQL in capital letters for readability.

Additional notes:

- Any data you create or change in Postgres will be available if you log into the same computer again at a later point.
- All SQL statements must end with a semicolon. The psql commands do not use a semicolon. If you enter an SQL command without a semicolon, or if you forget to close a pair of quotations, the line of text will be added to a buffer rather than being run. If this happens, use \r to clear your buffer and then re-enter your statement.
- If at any time a statement doesn't run or you get a strange error, a good first step is to try clearing the buffer with \r and then running the statement again.
- The psql interface keeps a history of your commands. You can scroll through that history with the up or down arrows in your keyboard and edit as needed.

How to Submit to Gradescope

There are three files to be submitted to Gradescope for this lab assignment:

Q1.sql

Q2.sql

Q3.sql

You may submit any number of files to the autograder, but you should submit all three files at the same time to receive full credit.

Here are some examples of submitting to Gradescope:

Each question is submitted in a separate .sql file.

You may submit the files one at a time or all together. Each file you submit will be autograded, and you will receive feedback as soon as you submit your file(s).

Example:

This is what happens upon submitting a correct Q1.sql file only:

Autograder Results

Autograder Output (hidden from students)	Total Points 2 / 6 pts
* Starting PostgreSQL 10 database server ...done.	Autograder Score 2.0 / 6.0
Query 1 (2/2)	Failed Tests Query 2 (0/2) Query 3 (0/2)
Query 2 (0/2)	Passed Tests Query 1 (2/2)
Test Failed: File not uploaded.	
Query 3 (0/2)	
Test Failed: Answer not submitted.	

Each question is submitted in a separate .sql file.
You may submit the files one at a time or all together. Each file you submit will be autograded, and you will receive feedback as soon as you submit your file(s).

Example:
This is what happens upon submitting an incorrect Q1.sql file.



Autograder Results

Autograder Output (hidden from students)
* Starting PostgreSQL 10 database server ...done.
Query 1 (0/2)
Test Failed: The query does not return the correct result.
Query 2 (0/2)
Test Failed: File not uploaded.
Query 3 (0/2)
Test Failed: Answer not submitted.

Total Points
0 / 6 pts
Autograder Score
0.0 / 6.0
Failed Tests
Query 1 (0/2)
Query 2 (0/2)
Query 3 (0/2)

Each question is submitted in a separate .sql file.
You may submit the files one at a time or all together. Each file you submit will be autograded, and you will receive feedback as soon as you submit your file(s).

Example:
This is what happens upon submitting all three correct files.

Autograder Results

Autograder Output (hidden from students)
* Starting PostgreSQL 10 database server ...done.
Query 1 (2/2)
Query 2 (2/2)
Query 3 (2/2)

Total Points
6 / 6 pts
Autograder Score
6.0 / 6.0
Passed Tests
Query 1 (2/2)
Query 2 (2/2)
Query 3 (2/2)